

# PS4

Sophia Chen

2026-02-07

**Due 02/07 at 5:00PM Central.**

“This submission is my work alone and complies with the 30538 integrity policy.” Add your initials to indicate your agreement: SC

## **Github Classroom Assignment Setup and Submission Instructions**

### **1. Accepting and Setting up the PS4 Assignment Repository**

- Each student must individually accept the repository for the problem set from Github Classroom (“ps4”) – <https://classroom.github.com/a/hWhcHqH>
  - You will be prompted to select your cnetid from the list in order to link your Github account to your cnetid.
  - If you can’t find your cnetid in the link above, click “continue to next step” and accept the assignment, then add your name, cnetid, and Github account to this Google Sheet and we will manually link it: <https://rb.gy/9u7fb6>
- If you authenticated and linked your Github account to your device, you should be able to clone your PS4 assignment repository locally.
- Contents of PS4 assignment repository:
  - `ps4_template.qmd`: this is the Quarto file with the template for the problem set. You will write your answers to the problem set here.

### **2. Submission Process:**

- Knit your completed solution `ps4.qmd` as a pdf `ps4.pdf`.
  - Your submission does not need runnable code. Instead, you will tell us either what code you ran or what output you got.
- To submit, push `ps4.qmd` and `ps4.pdf` to your PS4 assignment repository. Confirm on Github.com that your work was successfully pushed.

## Grading

- You will be graded on what was last pushed to your PS4 assignment repository before the assignment deadline
- Problem sets will be graded for completion as: {missing (0%); - (incomplete, 50%); + (excellent, 100%)}
  - The percent values assigned to each problem denote how long we estimate the problem will take as a share of total time spent on the problem set, not the points they are associated with.
- In order for your submission to be considered complete, you need to push both your `ps4.qmd` and `ps4.pdf` to your repository. Submissions that do not include both files will automatically receive 50% credit.

```

import pandas as pd
import altair as alt
import time

import warnings
warnings.filterwarnings('ignore')
alt.renderers.enable("png")

```

```

RendererRegistry.enable('png')

```

## Step 1: Develop initial scraper and crawler

```

import re
from urllib.parse import urljoin

import requests
import pandas as pd
from bs4 import BeautifulSoup

# target page
URL = "https://oig.hhs.gov/fraud/enforcement/"
BASE = "https://oig.hhs.gov"

# headers with user-agent
headers = {
    "User-Agent": "DAP30538CourseBot/1.0 (yanjiec@uchicago.edu)"
}

# get the page
resp = requests.get(URL, headers=headers, timeout=30)
resp.raise_for_status()

# parse the page using BeautifulSoup
soup = BeautifulSoup(resp.text, "lxml")

# types in the page "Type" filter
KNOWN_TYPES = {
    "Child Support",
    "CIA Reportable Events",
    "CMP and Affirmative Exclusions",
    "COVID-19",

```

```

    "Criminal and Civil Actions",
    "EMTALA/Patient Dumping",
    "Fraud Self-Disclosures",
    "Grant and Contractor Fraud Self-Disclosures",
    "State Enforcement Agencies",
    "Stipulated Penalties and Material Breaches",
}

# I asked ChatGPT how to extract the date from the page, and it suggested
↪ using a regex by first finding the months in the page text and then
↪ compile the regex for the date using the syntax of the regex module in
↪ Python
# regex for date
MONTHS =
    ↪ r"(January|February|March|April|May|June|July|August|September|October|November|December)"
DATE_RE = re.compile(rf"{MONTHS}\s+\d{{1,2}},\s+\d{{4}}")

# initialize empty list to store the data
rows = []

# I asked ChatGPT how to write <a href> in the regex, and it suggested using
↪ a[href]
# action title in <h2>, <h2> contains <a href>
# find all <h2> that contains <a href>
h2_links = soup.select("h2 a[href]")
for a in h2_links:
    title = a.get_text(" ", strip=True)
    # connect the base URL with the link
    link = urljoin(BASE, a.get("href"))

# I asked ChatGPT how to find the parent container, and it suggested
↪ using the find_parent method
# find the parent container
container = a.find_parent(["li", "div"])
if container is None:
    continue

# I asked ChatGPT how to join the text in the container, and it suggested
↪ using the strip_strings method
# join the text in the container
blob = " ".join(container.strip_strings)
# print(blob)

```

```

# find the date: Month Day, Year in the container text
m = DATE_RE.search(blob)
# print(m)
# I asked ChatGPT how to extract the date from a regex search, and it
↪ suggested using the group method
# extract the date from the container text
date = m.group(0) if m else None

# category: find the types in the container text
cats = []
for t in KNOWN_TYPES:
    if t in blob:
        cats.append(t)

# remove duplicates and sort by the order of appearance in the blob
if cats:
    cats = sorted(set(cats), key=lambda x: blob.find(x))
    category = "; ".join(cats)
else:
    category = None

# only collect the items that have date and at least one category
if date and category:
    rows.append(
        {
            "title": title,
            "date": date,
            "category": category,
            "link": link,
        }
    )

# only collect 20 items on the first page
if len(rows) >= 20:
    break

# convert to dataframe
df = pd.DataFrame(rows)

# print dataframe heads
print(df.head())

```

|   | title                                              | date \           |
|---|----------------------------------------------------|------------------|
| 0 | Houston Transplant Doctor Indicted For Making ...  | February 5, 2026 |
| 1 | MultiCare Health System to Pay Millions to Set...  | February 4, 2026 |
| 2 | Brooklyn Banker Pleads Guilty to Laundering Pr...  | February 3, 2026 |
| 3 | Delafield Man Sentenced to 18 Months' Imprison...  | February 3, 2026 |
| 4 | Former NFL Player Convicted for \$197M Medicare... | February 3, 2026 |

|   | category \                 |
|---|----------------------------|
| 0 | Criminal and Civil Actions |
| 1 | Criminal and Civil Actions |
| 2 | COVID-19                   |
| 3 | Criminal and Civil Actions |
| 4 | Criminal and Civil Actions |

|   | link                                                                                                              |
|---|-------------------------------------------------------------------------------------------------------------------|
| 0 | <a href="https://oig.hhs.gov/fraud/enforcement/houston-...">https://oig.hhs.gov/fraud/enforcement/houston-...</a> |
| 1 | <a href="https://oig.hhs.gov/fraud/enforcement/multicar...">https://oig.hhs.gov/fraud/enforcement/multicar...</a> |
| 2 | <a href="https://oig.hhs.gov/fraud/enforcement/brooklyn...">https://oig.hhs.gov/fraud/enforcement/brooklyn...</a> |
| 3 | <a href="https://oig.hhs.gov/fraud/enforcement/delafiel...">https://oig.hhs.gov/fraud/enforcement/delafiel...</a> |
| 4 | <a href="https://oig.hhs.gov/fraud/enforcement/former-n...">https://oig.hhs.gov/fraud/enforcement/former-n...</a> |

## Step 2: Making the scraper dynamic

### 1. Turning the scraper into a function

- a. Pseudo-Code

```

FUNCTION scrape_enforcement_actions(month, year, run_scraper):
BEGIN
  IF year < 2013 THEN
    PRINT "Please restrict to year >= 2013..."
    RETURN None
  END IF

  SET outfile TO "enforcement_actions_year_month.csv"

  IF run_scraper IS FALSE THEN
    IF outfile EXISTS THEN
      READ outfile AS dataframe
      RETURN dataframe
    ELSE
      PRINT "run_scraper=False and no cached csv found..."
    
```

```

        RETURN empty dataframe
    END IF
END IF

SET start_date TO first day of (year, month)
SET base_url TO "https://oig.hhs.gov"
SET current_url TO "https://oig.hhs.gov/fraud/enforcement/"
SET results TO empty list

WHILE current_url IS NOT None DO:
    REQUEST current_url (with headers)
    PARSE html WITH BeautifulSoup

    EXTRACT all enforcement action items on this page
    FOR EACH item DO:
        PARSE title, link, date, category FROM item
        CONVERT date string TO date object

        IF item_date >= start_date THEN
            APPEND {title, date, category, link} TO results
        END IF
    END FOR

    IF oldest date on this page < start_date THEN
        BREAK
    END IF

    FIND next page link
    IF next link EXISTS THEN
        SET current_url TO absolute next link
        SLEEP 1 second
    ELSE
        SET current_url TO None
    END IF
END WHILE

CONVERT results TO dataframe
SAVE dataframe TO outfile AS csv
RETURN dataframe
END
END FUNCTION

```

- b. Create Dynamic Scraper

```

import re
import time
from datetime import datetime, date
from urllib.parse import urljoin

import requests
import pandas as pd
from bs4 import BeautifulSoup

# same setup as Step 1
URL = "https://oig.hhs.gov/fraud/enforcement/"
BASE = "https://oig.hhs.gov"

headers = {
    "User-Agent": "DAP30538CourseBot/1.0 (yanjiec@uchicago.edu)"
}

KNOWN_TYPES = {
    "Child Support",
    "CIA Reportable Events",
    "CMP and Affirmative Exclusions",
    "COVID-19",
    "Criminal and Civil Actions",
    "EMTALA/Patient Dumping",
    "Fraud Self-Disclosures",
    "Grant and Contractor Fraud Self-Disclosures",
    "State Enforcement Agencies",
    "Stipulated Penalties and Material Breaches",
}

MONTHS =
↪ r"(January|February|March|April|May|June|July|August|September|October|November|December)"
DATE_RE = re.compile(rf"{MONTHS}\s+\d{{1,2}},\s+\d{{4}}")

# convert 'Month Day, Year' to datetime.date
def _parse_date_str(date_str: str) -> date | None:
    if not date_str:
        return None
    return datetime.strptime(date_str, "%B %d, %Y").date()

# find the next page url
def _find_next_page_url(soup: BeautifulSoup, current_url: str) -> str | None:
    a = soup.select_one("ul.pagination li.next-page a.pagination-next[href]")

```



```

    if a and a.get("href"):
        return urljoin(current_url, a["href"])

    a = soup.select_one("ul.pagination li.next-page a[href]")
    if a and a.get("href"):
        return urljoin(current_url, a["href"])

    return None

# scrape the enforcement actions
def scrape_enforcement_actions(month: int, year: int, run_scraper: bool =
↳ False) -> pd.DataFrame | None:
    if year < 2013:
        print("Please restrict to year >= 2013, since only enforcement
↳ actions after 2013 are listed.")
        return None

    start_date = date(year, month, 1)
    outfile = f"enforcement_actions_{year}_{month:02d}.csv"

    if not run_scraper:
        try:
            df_cached = pd.read_csv(outfile)
            # parse date column back to datetime.date
            df_cached["date"] = pd.to_datetime(df_cached["date"],
↳ errors="coerce")
            print(f"run_scraper=False, so loaded cached csv: {outfile}")
            return df_cached
        except FileNotFoundError:
            print(f"run_scraper=False and no cached csv found: {outfile}")
            return pd.DataFrame(columns=["title", "date", "category",
↳ "link"])

    # initialize empty list to store the data
    results = []
    current_url = URL

    while current_url is not None:
        resp = requests.get(current_url, headers=headers, timeout=30)
        resp.raise_for_status()
        soup = BeautifulSoup(resp.text, "lxml")

```

```

# use the same selection rule as Step 1
h2_links = soup.select("h2 a[href]")

# track the oldest date on this page
page_dates = []

for a in h2_links:
    title = a.get_text(" ", strip=True)
    link = urljoin(BASE, a.get("href"))

    container = a.find_parent(["li", "div"])
    if container is None:
        continue

    blob = " ".join(container.stripped_strings)

    # date by regex
    m = DATE_RE.search(blob)
    date_str = m.group(0) if m else None
    d = _parse_date_str(date_str)

    # category by matching KNOWN_TYPES
    cats = []
    for t in KNOWN_TYPES:
        if t in blob:
            cats.append(t)

    if cats:
        cats = sorted(set(cats), key=lambda x: blob.find(x))
        category = "; ".join(cats)
    else:
        category = None

    # only keep valid items
    if d and category:
        page_dates.append(d)

    # keep only since start_date
    if d >= start_date:
        results.append(
            {
                "title": title,
                "date": d.isoformat(),    # save as ISO string
            }
        )

```

```

        "category": category,
        "link": link,
    }
)

# if the oldest date on this page is already older than start_date,
↪ stop
if page_dates and min(page_dates) < start_date:
    break

# go next page
next_url = _find_next_page_url(soup, current_url)
if next_url:
    time.sleep(1)
    current_url = next_url
else:
    current_url = None

# convert to dataframe and drop duplicates
df = pd.DataFrame(results).drop_duplicates()
df.to_csv(outfile, index=False)
print(f"Saved: {outfile} (rows={len(df)})")
return df

# run the scraper since Jan 2024
df_2024 = scrape_enforcement_actions(month=1, year=2024, run_scraper=False)

# print the number of enforcement actions
print("Total enforcement actions since Jan 2024:", len(df_2024))

# print the earliest date and details
df_2024["date"] = pd.to_datetime(df_2024["date"], errors="coerce")
earliest_date = df_2024["date"].min()
print("Earliest date scraped:", earliest_date.date())

earliest_rows = (
    df_2024[df_2024["date"] == earliest_date]
    .sort_values(["title"])
    .reset_index(drop=True)
)

print("\nEarliest enforcement action(s) details:")
print(earliest_rows[["title", "date", "category", "link"]])

```

```
run_scraper=False, so loaded cached csv: enforcement_actions_2024_01.csv
Total enforcement actions since Jan 2024: 1787
Earliest date scraped: 2024-01-03
```

Earliest enforcement action(s) details:

|   | title                                             | date       | \ |
|---|---------------------------------------------------|------------|---|
| 0 | Former Nurse Aide Indicted In Death Of Clarksv... | 2024-01-03 |   |
| 1 | Laredo Resident Admits To Impersonating Licens... | 2024-01-03 |   |

|   | category                   | \ |
|---|----------------------------|---|
| 0 | State Enforcement Agencies |   |
| 1 | Criminal and Civil Actions |   |

|   | link                                                                                                              |
|---|-------------------------------------------------------------------------------------------------------------------|
| 0 | <a href="https://oig.hhs.gov/fraud/enforcement/former-n...">https://oig.hhs.gov/fraud/enforcement/former-n...</a> |
| 1 | <a href="https://oig.hhs.gov/fraud/enforcement/laredo-r...">https://oig.hhs.gov/fraud/enforcement/laredo-r...</a> |

- c. Test Your Code

```
import re
import time
from datetime import datetime, date
from urllib.parse import urljoin

import requests
import pandas as pd
from bs4 import BeautifulSoup

# same setup as Step 1
URL = "https://oig.hhs.gov/fraud/enforcement/"
BASE = "https://oig.hhs.gov"

headers = {
    "User-Agent": "DAP30538CourseBot/1.0 (yanjiec@uchicago.edu)"
}

KNOWN_TYPES = {
    "Child Support",
    "CIA Reportable Events",
    "CMP and Affirmative Exclusions",
    "COVID-19",
    "Criminal and Civil Actions",
```

```

    "EMTALA/Patient Dumping",
    "Fraud Self-Disclosures",
    "Grant and Contractor Fraud Self-Disclosures",
    "State Enforcement Agencies",
    "Stipulated Penalties and Material Breaches",
}

MONTHS =
↪ r"(January|February|March|April|May|June|July|August|September|October|November|December)"
DATE_RE = re.compile(rf"{MONTHS}\s+\d{{1,2}},\s+\d{{4}}")

# convert 'Month Day, Year' to datetime.date
def _parse_date_str(date_str: str) -> date | None:
    if not date_str:
        return None
    return datetime.strptime(date_str, "%B %d, %Y").date()

# find the next page url
def _find_next_page_url(soup: BeautifulSoup, current_url: str) -> str | None:
    a = soup.select_one("ul.pagination li.next-page a.pagination-next[href]")
    if a and a.get("href"):
        return urljoin(current_url, a["href"])

    a = soup.select_one("ul.pagination li.next-page a[href]")
    if a and a.get("href"):
        return urljoin(current_url, a["href"])

    return None

# scrape the enforcement actions
def scrape_enforcement_actions(month: int, year: int, run_scraper: bool =
↪ False) -> pd.DataFrame | None:
    if year < 2013:
        print("Please restrict to year >= 2013, since only enforcement
↪ actions after 2013 are listed.")
        return None

    start_date = date(year, month, 1)
    outfile = f"enforcement_actions_{year}_{month:02d}.csv"

    if not run_scraper:
        try:
            df_cached = pd.read_csv(outfile)

```

```

        # parse date column back to datetime.date
        df_cached["date"] = pd.to_datetime(df_cached["date"],
↪ errors="coerce")
        print(f"run_scraper=False, so loaded cached csv: {outfile}")
        return df_cached
    except FileNotFoundError:
        print(f"run_scraper=False and no cached csv found: {outfile}")
        return pd.DataFrame(columns=["title", "date", "category",
↪ "link"])

# initialize empty list to store the data
results = []
current_url = URL

while current_url is not None:
    resp = requests.get(current_url, headers=headers, timeout=30)
    resp.raise_for_status()
    soup = BeautifulSoup(resp.text, "lxml")

    # use the same selection rule as Step 1
    h2_links = soup.select("h2 a[href]")

    # track the oldest date on this page
    page_dates = []

    for a in h2_links:
        title = a.get_text(" ", strip=True)
        link = urljoin(BASE, a.get("href"))

        container = a.find_parent(["li", "div"])
        if container is None:
            continue

        blob = " ".join(container.stripped_strings)

        # date by regex
        m = DATE_RE.search(blob)
        date_str = m.group(0) if m else None
        d = _parse_date_str(date_str)

        # category by matching KNOWN_TYPES
        cats = []
        for t in KNOWN_TYPES:

```

```

        if t in blob:
            cats.append(t)

    if cats:
        cats = sorted(set(cats), key=lambda x: blob.find(x))
        category = "; ".join(cats)
    else:
        category = None

    # only keep valid items
    if d and category:
        page_dates.append(d)

    # keep only since start_date
    if d >= start_date:
        results.append(
            {
                "title": title,
                "date": d.isoformat(),    # save as ISO string
                "category": category,
                "link": link,
            }
        )

    # if the oldest date on this page is already older than start_date,
    ↪ stop
    if page_dates and min(page_dates) < start_date:
        break

    # go next page
    next_url = _find_next_page_url(soup, current_url)
    if next_url:
        time.sleep(1)
        current_url = next_url
    else:
        current_url = None

    # convert to dataframe and drop duplicates
    df = pd.DataFrame(results).drop_duplicates()
    df.to_csv(outfile, index=False)
    print(f"Saved: {outfile} (rows={len(df)})")
    return df

```

```

# run the scraper since Jan 2022
df_2022 = scrape_enforcement_actions(month=1, year=2022, run_scraper=False)

# print the number of enforcement actions
print("Total enforcement actions since Jan 2024:", len(df_2022))

# print the earliest date and details
df_2022["date"] = pd.to_datetime(df_2022["date"], errors="coerce")
earliest_date = df_2022["date"].min()
print("Earliest date scraped:", earliest_date.date())

earliest_rows = (
    df_2022[df_2022["date"] == earliest_date]
    .sort_values(["title"])
    .reset_index(drop=True)
)

print("\nEarliest enforcement action(s) details:")
print(earliest_rows[["title", "date", "category", "link"]])

```

run\_scraper=False, so loaded cached csv: enforcement\_actions\_2022\_01.csv

Total enforcement actions since Jan 2024: 3377

Earliest date scraped: 2022-01-04

Earliest enforcement action(s) details:

|   | title                                              | date       | \ |
|---|----------------------------------------------------|------------|---|
| 0 | Central Medical Systems, LLC, Alan Trent Harle...  | 2022-01-04 |   |
| 1 | Integrated Pain Management Medical Group Agree...  | 2022-01-04 |   |
| 2 | Ohio home healthcare provider agrees to pay \$5... | 2022-01-04 |   |

|   | category                   | \ |
|---|----------------------------|---|
| 0 | Criminal and Civil Actions |   |
| 1 | Fraud Self-Disclosures     |   |
| 2 | Criminal and Civil Actions |   |

|   | link                                                                                                              |
|---|-------------------------------------------------------------------------------------------------------------------|
| 0 | <a href="https://oig.hhs.gov/fraud/enforcement/central-...">https://oig.hhs.gov/fraud/enforcement/central-...</a> |
| 1 | <a href="https://oig.hhs.gov/fraud/enforcement/integrat...">https://oig.hhs.gov/fraud/enforcement/integrat...</a> |
| 2 | <a href="https://oig.hhs.gov/fraud/enforcement/ohio-hom...">https://oig.hhs.gov/fraud/enforcement/ohio-hom...</a> |



### Step 3: Plot data based on scraped data

#### 1. Plot the number of enforcement actions over time

```
import pandas as pd
import altair as alt

alt.data_transformers.disable_max_rows()

# read the cached csv
df = pd.read_csv("enforcement_actions_2022_01.csv")

# ISO dates
df["date"] = pd.to_datetime(df["date"], errors="coerce")

# aggregate to month-year
monthly = (
    df.assign(year_month=lambda x:
        ↪ x["date"].dt.to_period("M").dt.to_timestamp())
        .groupby("year_month", as_index=False)
        .size()
        .rename(columns={"size": "n_actions"})
        .sort_values("year_month")
)

# print the first few rows
print(monthly.head())

# make a line chart
chart = alt.Chart(monthly).mark_line().encode(
    x=alt.X(
        "year_month:T", title="Month-Year",
        axis=alt.Axis(
            format="%Y-%m",
            tickCount="month",
            tickMinStep=3 # every 3 months
        )
    ),
    y=alt.Y("n_actions:Q", title="Number of enforcement actions"),
    tooltip=[alt.Tooltip("year_month:T", title="Month",
        ↪ format="%Y-%m"),
        alt.Tooltip("n_actions:Q", title="Actions")]
```

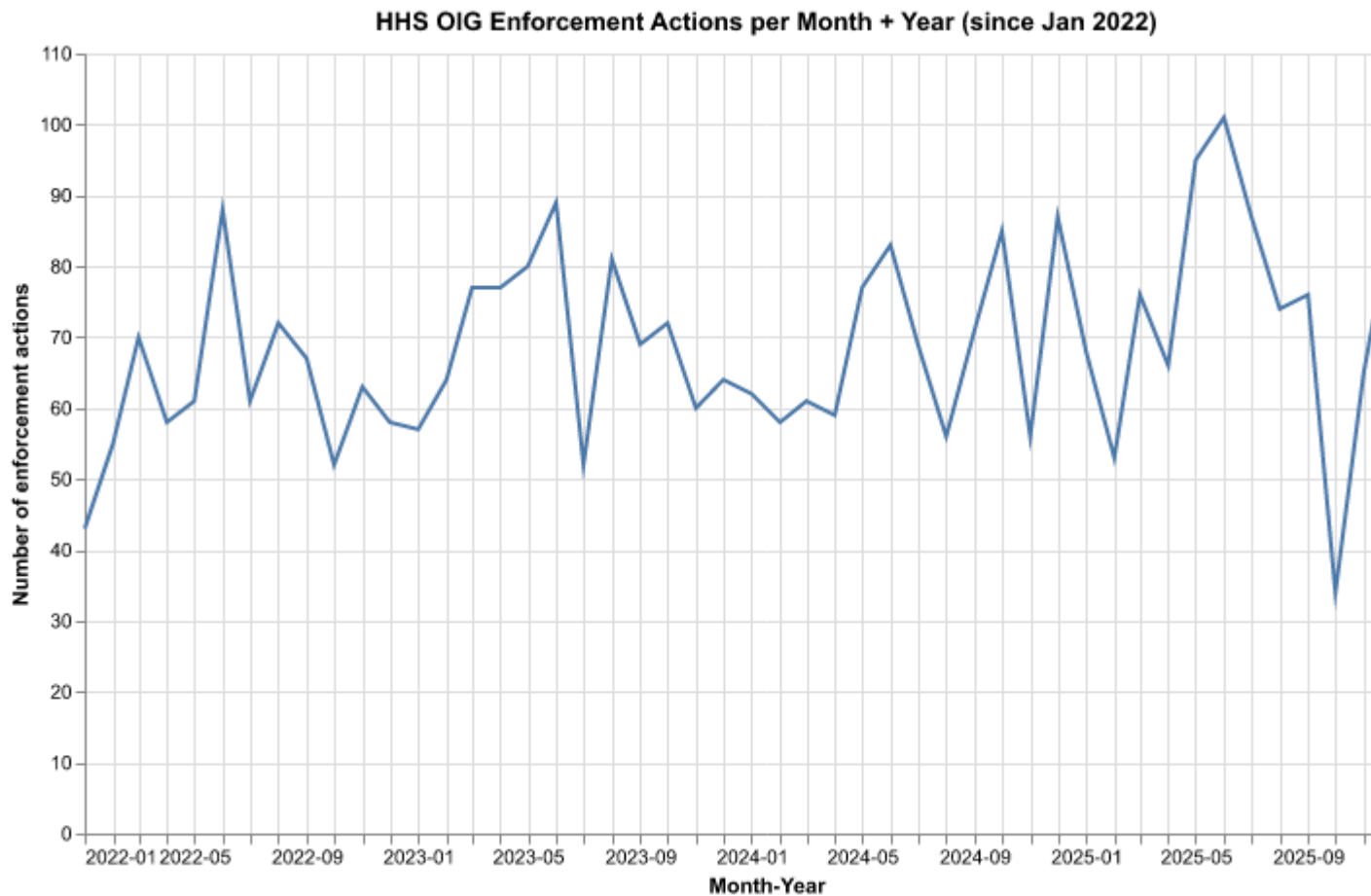
```

        ).properties(
            title="HHS OIG Enforcement Actions per Month + Year (since Jan
↪ 2022)",
            width=700,
            height=400
        )

chart.show()
chart.save("chart1.png", engine="vl-convert")

```

|   | year_month | n_actions |
|---|------------|-----------|
| 0 | 2022-01-01 | 43        |
| 1 | 2022-02-01 | 55        |
| 2 | 2022-03-01 | 70        |
| 3 | 2022-04-01 | 58        |
| 4 | 2022-05-01 | 61        |



![]chart1.png

## 2. Plot the number of enforcement actions categorized:

- based on “Criminal and Civil Actions” vs. “State Enforcement Agencies”

```
import pandas as pd
import altair as alt

alt.data_transformers.disable_max_rows()

# read the cached csv
df = pd.read_csv("enforcement_actions_2022_01.csv")

# ISO dates
```

```

df["date"] = pd.to_datetime(df["date"], errors="coerce")
df = df.dropna(subset=["date"])

# month-year to timestamp
df["year_month"] = df["date"].dt.to_period("M").dt.to_timestamp()

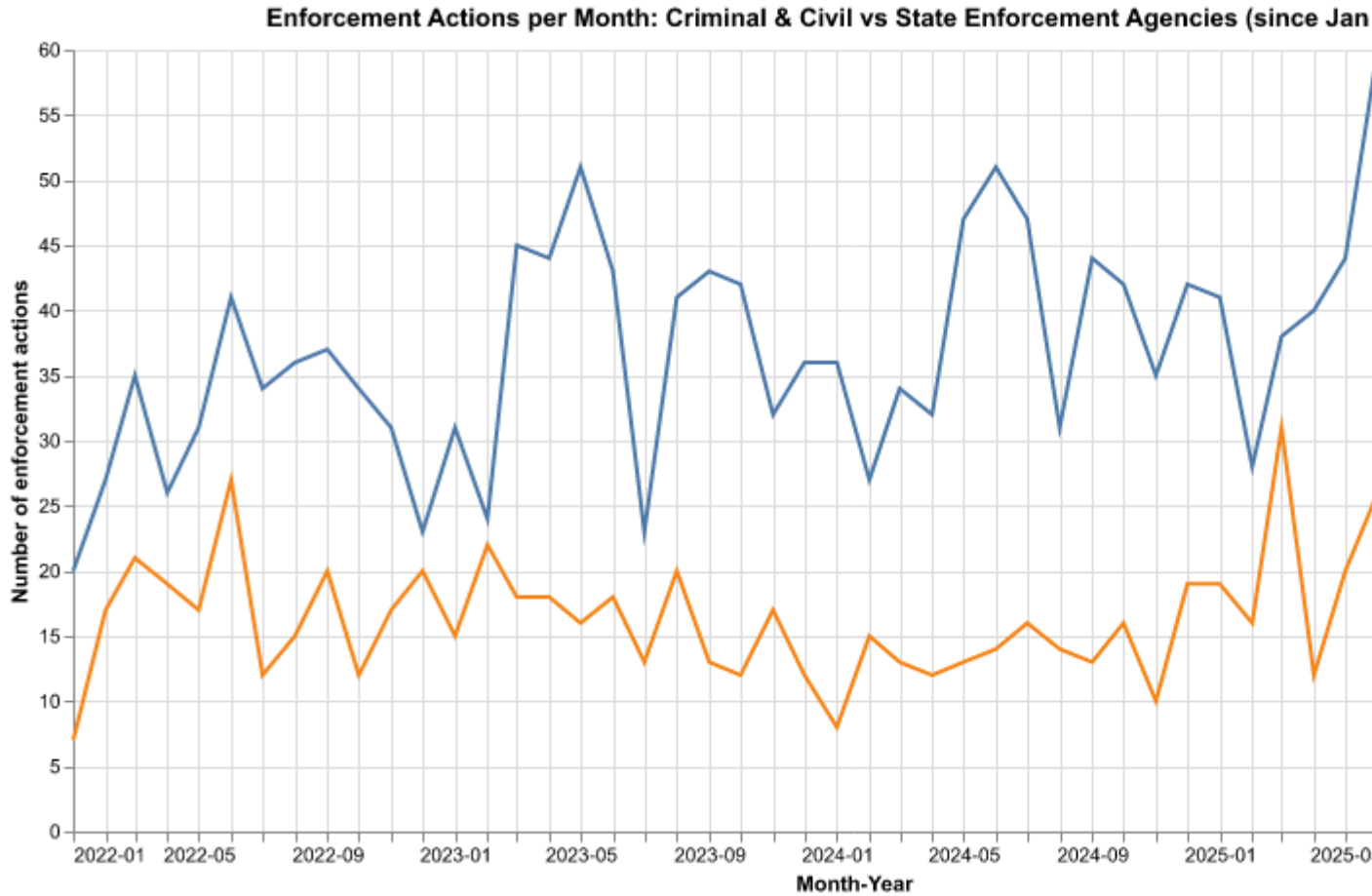
# the two categories to keep
cats_keep = ["Criminal and Civil Actions", "State Enforcement Agencies"]

# aggregate to month-year
monthly_cat = (
    df[df["category"].isin(cats_keep)]
    .groupby(["year_month", "category"], as_index=False)
    .size()
    .rename(columns={"size": "n_actions"})
    .sort_values(["year_month", "category"])
)

# make a line chart
chart_a = alt.Chart(monthly_cat).mark_line().encode(
    x=alt.X("year_month:T", title="Month-Year",
        axis=alt.Axis(format="%Y-%m", tickCount="month", tickMinStep=3)
    ),
    y=alt.Y("n_actions:Q", title="Number of enforcement actions"),
    color=alt.Color("category:N", title="Category"),
    tooltip=[
        alt.Tooltip("year_month:T", title="Month", format="%Y-%m"),
        alt.Tooltip("category:N", title="Category"),
        alt.Tooltip("n_actions:Q", title="Actions"),
    ],
).properties(
    title="Enforcement Actions per Month: Criminal & Civil vs State
↪ Enforcement Agencies (since Jan 2022)",
    width=800,
    height=400
)

chart_a.show()
chart_a.save("chart2.png", engine="vl-convert")

```



![]chart2.png

- based on five topics

```
import re

# classify the topics
# I asked ChatGPT to give me several words that are related to each topic,
# and then I used the regex to classify the topics.
def classify_topic(title: str) -> str:
    t = str(title).lower()

    # Bribery/Corruption
    if
        re.search(r"\b(bribe|bribery|kickback|corrupt|corruption|extortion|quid
        pro quo)\b", t):
```

```

        return "Bribery/Corruption"

# Drug Enforcement
if re.search(r"\b(drug|opioid|fentanyl|oxycodone|hydrocodone|pill
↳ mill|controlled substance|dea)\b", t):
    return "Drug Enforcement"

# Financial Fraud
if re.search(r"\b(bank|bankrupt|wire fraud|money
↳ laundering|launder|embezzle|investment|securities|loan|tax|irs|identity
↳ theft|credit|mortgage|financial)\b", t):
    return "Financial Fraud"

# Health Care Fraud
if re.search(r"\b(medicare|medicaid|health
↳ care|healthcare|claims?|billing|billings|provider|hospital|clinic|pharmacy|home
↳ health|hospice|dme|durable medical|lab|laboratory)\b", t):
    return "Health Care Fraud"

# Other
return "Other"

# filter the data to only include "Criminal and Civil Actions"
df_cc = df[df["category"] == "Criminal and Civil Actions"].copy()
# classify the topics
df_cc["topic"] = df_cc["title"].apply(classify_topic)

# aggregate to month-year
monthly_topic = (
    df_cc.groupby(["year_month", "topic"], as_index=False)
        .size()
        .rename(columns={"size": "n_actions"})
        .sort_values(["year_month", "topic"])
)

# the order of the topics in the legend
topic_order = ["Health Care Fraud", "Financial Fraud", "Drug Enforcement",
↳ "Bribery/Corruption", "Other"]

# make a line chart
chart_b = alt.Chart(monthly_topic).mark_line().encode(
    x=alt.X("year_month:T", title="Month-Year",

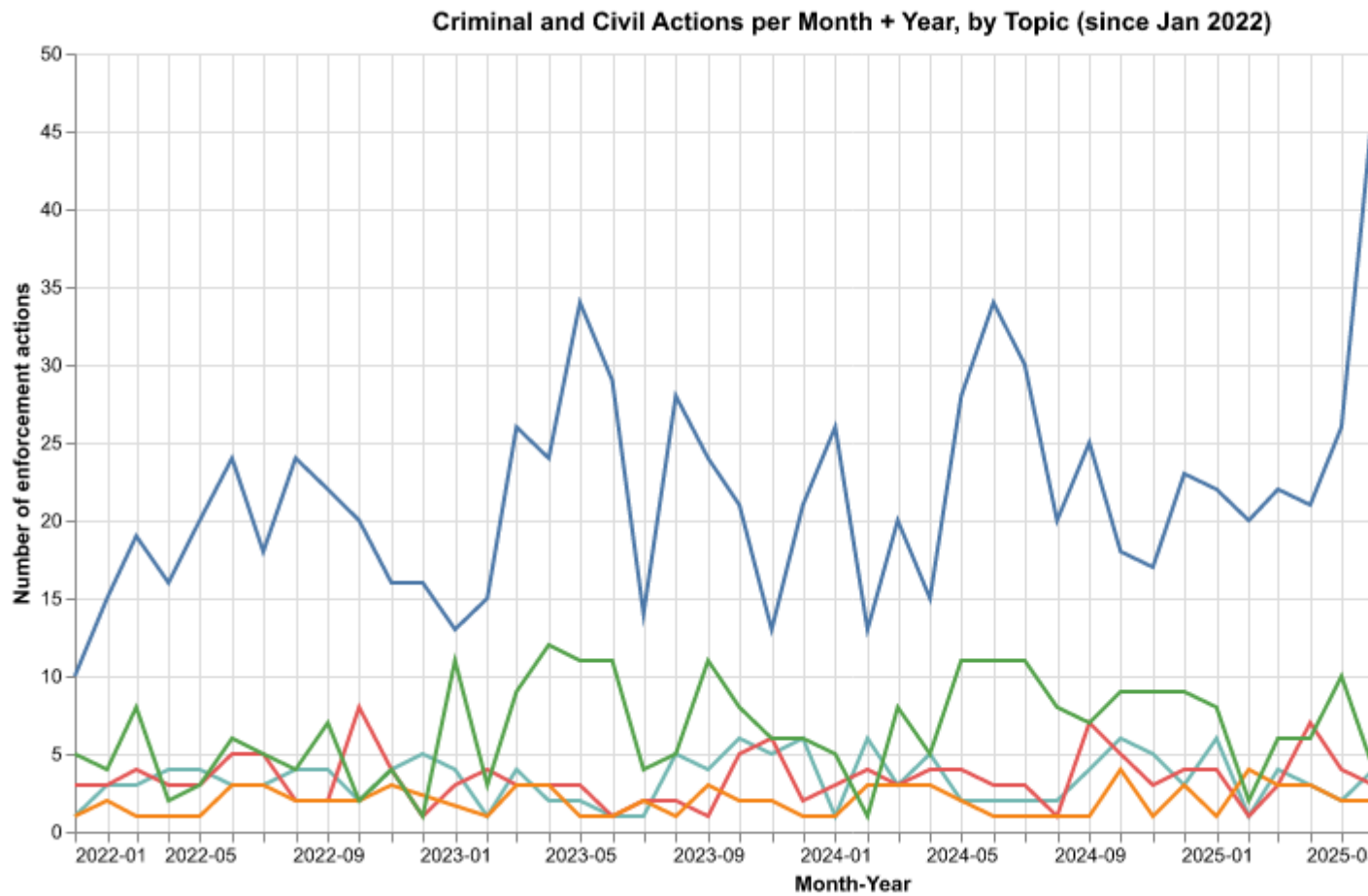
```

```

        axis=alt.Axis(format="%Y-%m", tickCount="month", tickMinStep=3)
    ),
    y=alt.Y("n_actions:Q", title="Number of enforcement actions"),
    color=alt.Color("topic:N", sort=topic_order, title="Topic"),
    tooltip=[
        alt.Tooltip("year_month:T", title="Month", format="%Y-%m"),
        alt.Tooltip("topic:N", title="Topic"),
        alt.Tooltip("n_actions:Q", title="Actions"),
    ],
    ).properties(
        title="Criminal and Civil Actions per Month + Year, by Topic (since
↪ Jan 2022)",
        width=800,
        height=400
    )

chart_b.show()
chart_b.save("chart3.png", engine="vl-convert")

```



![]chart3.png